

DAY-18 — Cryptography Basics

CyberNotes • Detailed Beginner Reference (Improved Edition)

Learning goal: Build a practical, working understanding of **cryptography basics**, connect the concept to real cybersecurity work, and practice it safely in a local lab.

Study rule: Understand a command or concept before copying it. Security tools are powerful because they interact with real systems — your practice environment should always be intentionally controlled and authorized.

1. Core Concepts

Core idea: Cryptography Basics is a building block for practical cybersecurity. This section explains what each term actually means, where you'll encounter it, and how it connects to the bigger picture.

Why it matters: Security work depends on understanding normal system and network behavior before you can recognize abnormal behavior. This topic connects directly to Linux, networking, web security, monitoring, and troubleshooting — the threads running through this whole 20-day series.

Vocabulary covered: plaintext, ciphertext, encoding vs encryption, hashing, symmetric/asymmetric crypto, signatures.

Reference	What to remember
Encoding	representation change, not secrecy
Hashing	integrity/fingerprint
AES	symmetric encryption
RSA/ECC	asymmetric cryptography
Digital signature	authenticity/integrity
TLS	protect network connections

2. Concept Explanations

Each term below is explained in plain language, with a note on why it matters for security work. Read these before running any commands.

1. plaintext

Plaintext is data in its original, readable form before any encryption is applied. Anywhere plaintext sensitive data (passwords, personal information) is stored or transmitted unnecessarily is a security risk, regardless of how strong the surrounding system's encryption elsewhere might be.

2. ciphertext

Ciphertext is the scrambled output produced by encrypting plaintext, unreadable without the correct decryption key. Seeing ciphertext tells you data was encrypted; it does not by itself tell you the encryption was implemented correctly or that the key was protected properly.

3. encoding vs encryption

Encoding (like Base64) simply changes how data is represented for compatibility purposes and provides zero secrecy — it's trivially reversible by anyone with no key required. Encryption, by contrast, requires a secret key to reverse. A very common real-world mistake is treating Base64-encoded data as if it were secured, when it is not.

4. hashing

Hashing produces a fixed-size fingerprint of data that's practically impossible to reverse and changes drastically with even a tiny change in the input, making it ideal for verifying integrity (did this file change?) and storing passwords (Day 17) — but unlike encryption, hashing is one-way and not meant to be "decrypted" back to the original.

5. symmetric/asymmetric crypto

Symmetric encryption (like AES) uses the same key to encrypt and decrypt, and is fast — but that key must be shared secretly beforehand. Asymmetric encryption (like RSA or ECC) uses a mathematically linked key pair — a public key anyone can use to encrypt, and a private key only the owner holds to decrypt — solving the key-sharing problem at the cost of more computation.

6. signatures

A digital signature uses a private key to "sign" data in a way that anyone with the corresponding public key can verify, proving both who sent it (authenticity) and that it wasn't altered in transit (integrity). This is a core building block of TLS certificates, which is why HTTPS provides both encryption and a verifiable identity for the server.

3. Safe Practical Lab

Step 1. Define the lab target and the exact question you are trying to answer.

Step 2. Run the smallest safe observation or command that can answer that question.

Step 3. Read the output carefully; record important fields, errors, timestamps, and assumptions.

Step 4. Change one variable at a time and compare the result.

Step 5. Write a short conclusion and clean up temporary files or services.

Example lab sequence:

```
Hash a sample string with sha256sum
compare two hashes
use an approved demo to observe encryption/decryption
record the difference between hash and encryption
```

4. Common Mistakes & Troubleshooting

- Running a command without checking its target, current directory, or privileges.
- Confusing a tool's output with proof of a vulnerability or compromise — output is evidence, not a conclusion.
- Ignoring error messages instead of reading the exact reason for failure.
- Changing several settings at once, making it hard to tell what actually fixed or broke the lab.
- Saving real credentials, tokens, or personal data in practice notes or public repositories.
- Testing a public target just because it's reachable — reachability is never the same as authorization.

Troubleshooting checklist:

- What system am I on?
- What exact command/request did I run?
- What did I expect?
- What happened instead?
- What does the error message say?
- Can I reproduce it in a clean local lab?
- What changed since the last successful attempt?

5. Deep-Dive Questions

Answer these in your own words after finishing the lab — they're designed to check real understanding, not memorization.

Q1. What problem does plaintext solve, in your own words?

Notes: _____

Q2. What is the difference between normal behavior and a suspicious observation in this topic?

Notes: _____

Q3. Which command, log, packet, file, or response would you inspect first, and why?

Notes: _____

Q4. What could cause a misleading result or false positive here?

Notes: _____

Q5. How would you document your observation so another learner could reproduce it safely?

Notes: _____

Q6. Which security control reduces the risk associated with this topic?

Notes: _____

6. Revision Checklist

- I can explain cryptography basics in my own words.
- I can identify the main terms without looking at the notes.
- I can perform the example in a controlled lab.
- I can interpret the output instead of only copying it.
- I know at least one defensive or troubleshooting use for the concept.
- I can document the result clearly and safely.

7. Safety & Scope

Use these commands and tools only on systems you own, a local virtual machine, a CTF, an intentionally vulnerable application, or a target for which you have explicit written permission.

Keep a written scope: target, allowed actions, time window, and stop conditions. Never scan or test random public systems.

When practicing, save your notes and outputs so you can explain what you observed rather than blindly running commands.

Next step: After Day-18, review your notes and connect this topic to the next subject in the series. The goal is not to memorize commands — it's to understand systems well enough to reason about security.

8. Quick Reference & Personal Notes

Topic: Day-18 — Cryptography Basics

Three things I learned:

1. _____
1. _____
1. _____

Commands / concepts to memorize:

- Encoding — representation change, not secrecy
- Hashing — integrity/fingerprint
- AES — symmetric encryption
- RSA/ECC — asymmetric cryptography
- Digital signature — authenticity/integrity
- TLS — protect network connections

My lab result

What I tested: _____

What I observed: _____

What I would investigate next: _____

Final reminder: A strong cybersecurity learner can explain both the offensive concept and the defensive reason for learning it. Keep practice authorized, reproducible, and documented.